Linux Superpowers

Using eBPF to Bend the Kernel to Your Will

Luis Jira (Red Hat) / VIScon 2026

WHOAMI

Luis Jira

Software Engineer @ Red Hat

ACADEMIC BACKGROUND

Computer Science

- ETH Zürich
  - ex-VIScon organizer
- KTH Royal Institute of Technology

CURRENT ROLE

Red Hat

Network Observability for OpenShift & Kubernetes

WHY THIS TALK?

I like eBPF

- First contact during Master's thesis
- Now working with eBPF
- Exciting future ahead

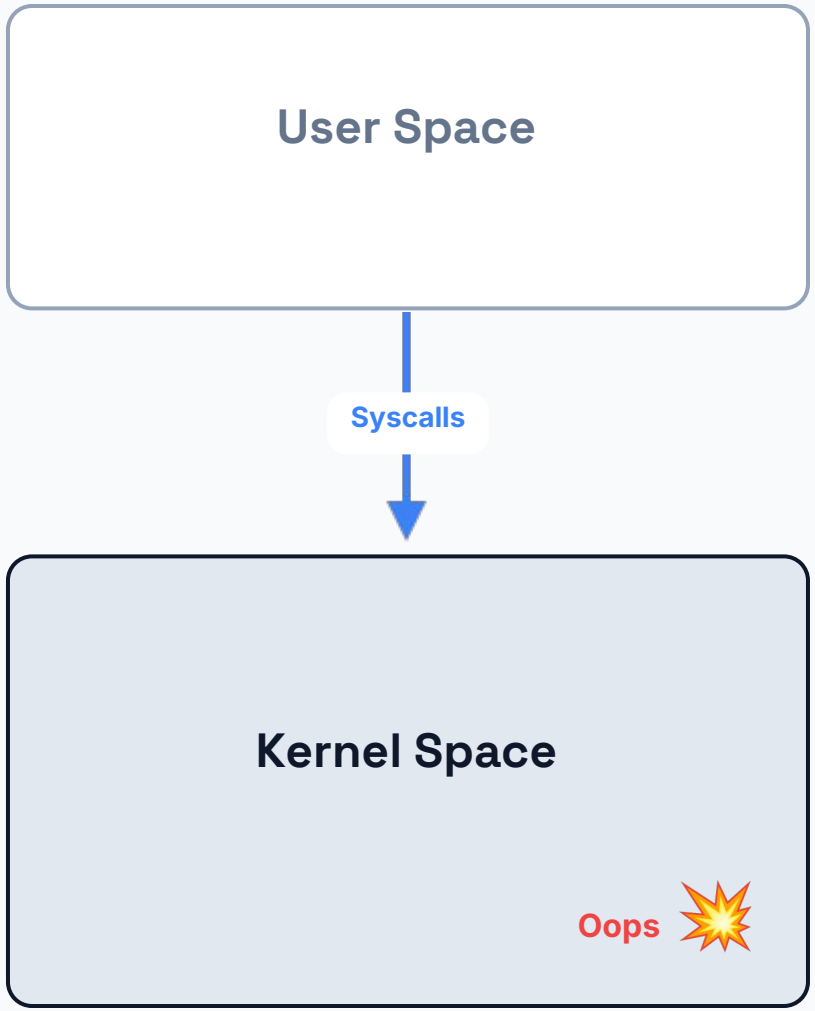
ROADMAP

The Path Ahead



The Privilege Boundary

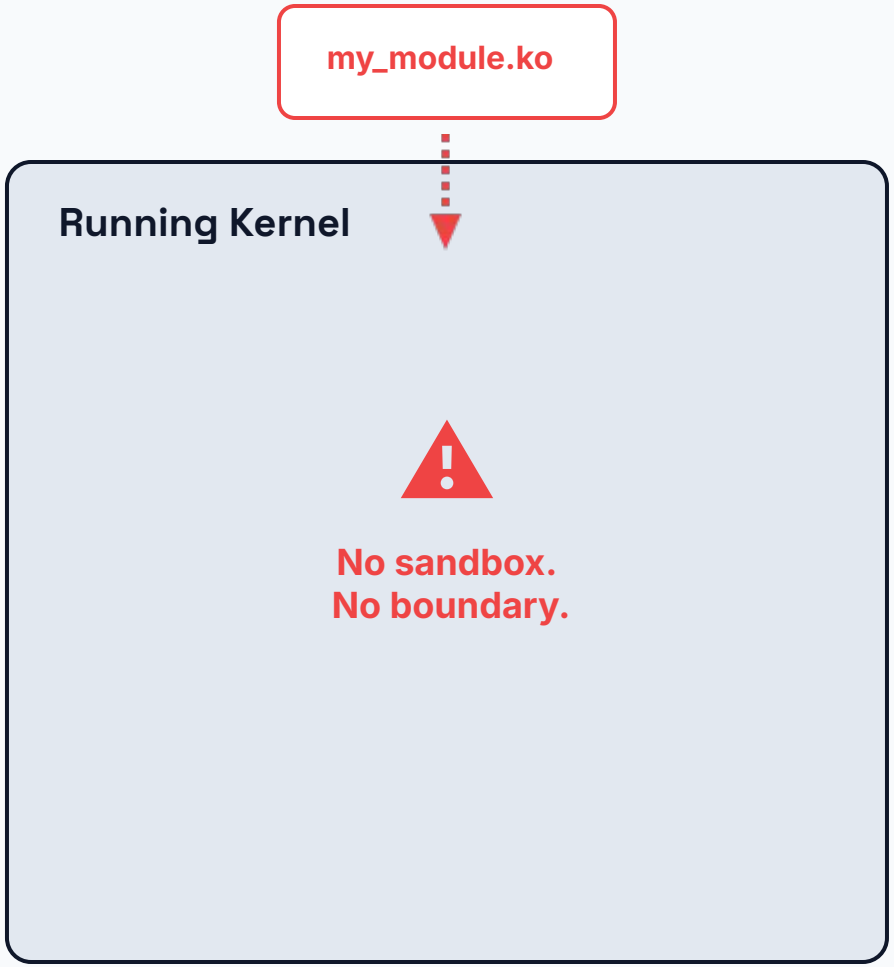
- **User space:**unprivileged, isolated processes
- **Kernel space:** shared, privileged, global failure
- **Syscalls:** the heavily controlled interface
- **Core tension:** absolute power vs total safety



LEGACY EXTENSION

Kernel Modules

- Load full C into running kernel
- Complete, unrestricted access to internals
- Fragile ABIs and constant rebuilds required
- **Power without a safety contract**



DESIGN GOALS

The Ultimate Wish List

- [x] Custom logic executed directly on hot paths.
- [x] Load and unload without rebooting.
- [x] Safety guarantees
- [x] Curated maps and helpers (no raw pointers).
- [x] Fully controllable from user space.



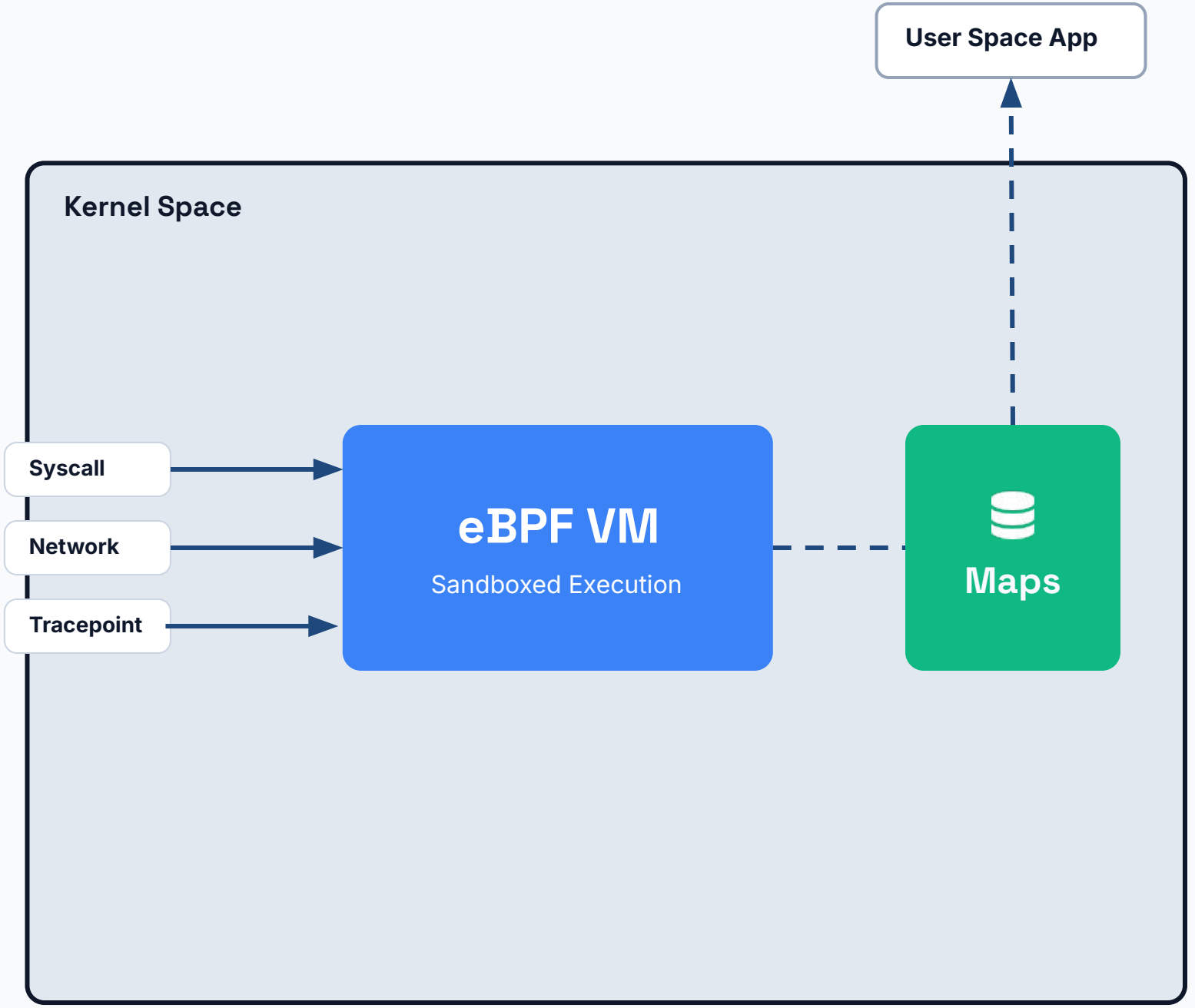
DEFINITION

What is eBPF?

A bytecode VM inside the Linux kernel.

Programs run precisely at event hook points.

Maps maintain state; helpers provide capabilities.



CAPABILITIES

Core Strengths

Dynamic Attach

Load and unload instantly without disruption.

JIT Compiled

Executes at near-native speed.

Rich State

Hash, array, ringbuf, and LRU maps.

CO-RE Portability

Compile once, run on any modern kernel.

No Module Tax

Extend the OS without writing a kernel module.

ECOSYSTEM

Use Cases

Observability

bpftrace, continuous profilers.

Identify latency without overhead.

Networking

XDP, TC, Cilium dataplanes.

High-performance routing and load balancing.

Security

Tetragon, Falco-class policies.

Strict runtime enforcement at the syscall layer.

Performance

sched_ext (custom schedulers).

Explain and tune CPU behavior dynamically.

How is this safe?

- Custom code in the kernel
- Widely deployed in production

THE CHALLENGE

The Safety Problem

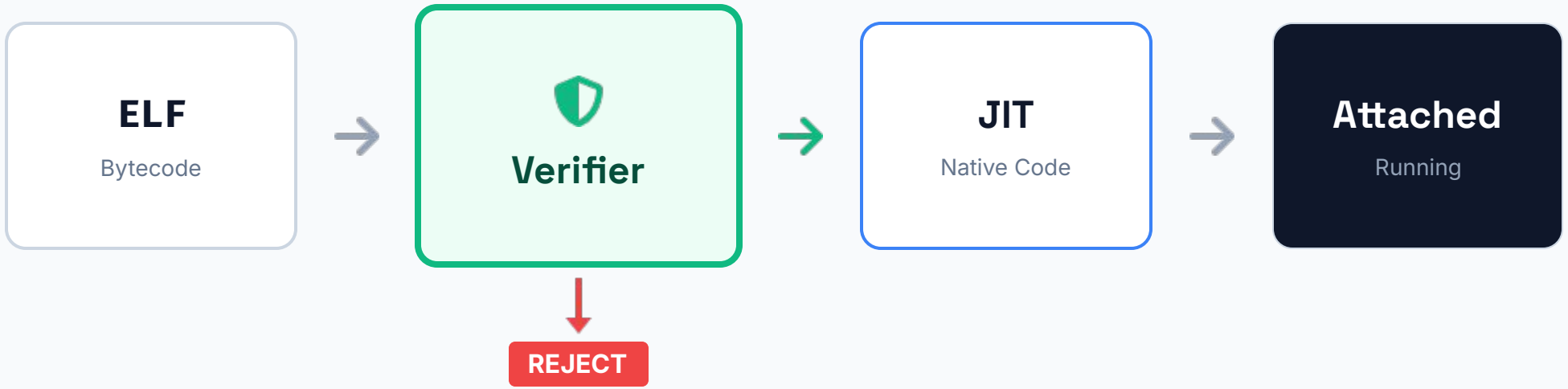
- Non-termination
- Invalid memory access
- Resource abuse
- **Goal:** Don’t trust the program

```
void bad_idea(void *ctx) { char *ptr = (char *)0xffff...;
while(1) { *ptr = 0; } }
```

THE SOLUTION

Enter The Verifier

Static analysis at load time



STATIC ANALYSIS

What The Verifier Checks

- Control-flow graph exploration
- Register types & value ranges
- Memory bounds
- Helper call contracts
- Hard complexity & stack budgets

THE API

Maps & Helpers

Helpers

Curated kernel calls

`bpf_get_current_pid_tgid()`

`bpf_ktime_get_ns()`

`bpf_redirect()`



Maps

Shared state

Arrays

Hash Maps

Ringbufs

⇄ USER SPACE BRIDGE

REALITY CHECK

Limits

Verifier Guarantees

- Kernel will not panic.
- Memory is accessed safely.
- Execution will terminate.

Still Your Problem

- Safe execution ≠ correct policy.
- Valid programs may need reshaping.
- Complexity limits are real constraints.
- CAP_BPF privilege is still required.

HOOKS

Attachment Is The Product

OBSERVABILITY & TRACING

System Introspection

kprobe Dynamic

tracepoint Static

uprobe User space

SECURITY & CONTROL

Runtime Governance

LSM Security

cgroup Containers

NETWORKING

Packet Processing

XDP Early Ingress

TC Traffic Control

PERSONALITIES

Tracing vs Networking

Tracing / Observability

Observe & aggregate. Rarely mutate.

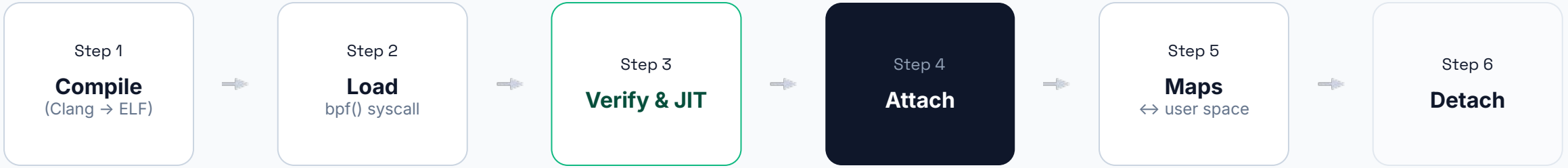
Networking / Policy

Decide fate: drop, redirect, rewrite.



WORKFLOW

The eBPF Lifecycle



STRUCTURE

Anatomy of a Program

- SEC("...") — defines attach intent
- Context struct — hook-specific view
- Logic — restricted helpers + maps
- License string — for verifier validation

```
SEC("tc")
int drop_all(struct __sk_buff *ctx) {
    return TC_ACT_SHOT;
}
char _license[] SEC("license") = "GPL";
```

DEMO SETUP

DNS Refusal Pipeline



LIVE DEMO ENVIRONMENT

Client / Server Netns Architecture & eBPF Hook Setup

netns client

Interface: veth-c
IPv6 Address: `fd00::53:1`

DNS Query

```
$ dig -6 @fd00:53::2 demo.viscon AAAA
```

eBPF Program (tc hook)

Attached directly to egress hook on `veth-c`.

- Detects outgoing DNS queries on Port 53.
- Swaps MACs, IPv6, & UDP ports inline.
- Rewrites query into response with **REFUSED** status.

netns server

Interface: veth-s
IPv6 Address: `fd00:53::2`

DNS Server
Always responds fd00:53::cafe

Listens on: `fd00:53::2:53`

LIVE ACTION

Bend DNS to Your Will

DEBRIEF

What Just Happened?

01

Attach

TC egress

02

Safety

Verifier bounds checked

03

State

Map counter incremented

04

Action

csum + bpf_redirect

SUMMARY

Takeaways

01 **Modules vs eBPF:** Power without contract vs Programmable proof.

02 **The API:** Curated events + Maps + Helpers.

03 **The Verifier:** Load-time safety proofs.

04 **Attachment:** Hook location chooses your superpower.

05 **Demo Reality:** DNS refused in-kernel safely.

Thank You!

Questions?

---

github.com/luisjira/ebpf-demo

Backup slides

More details

`github.com/luisjira/ebpf-demo`

eBPF PACKET MANIPULATION

DNS Query Refusal & Ingress Redirect

```
flags |= DNS_QR_BIT;
flags = (flags & ~0x000F) | DNS_RCODE_REFUSED;
update_udp_csum_for_flags(udp, dns, bpf_htons(flags));

/* Swap MACs so frame looks like peer origin */
unsigned char tmp_mac[ETH_ALEN];
__builtin_memcpy(tmp_mac, eth->h_dest, ETH_ALEN);
__builtin_memcpy(eth->h_dest, eth->h_source, ETH_ALEN);
__builtin_memcpy(eth->h_source, tmp_mac, ETH_ALEN);

/* Swap IPv6 addresses */
struct in6_addr tmp_ip6;
__builtin_memcpy(&tmp_ip6, &ip6->saddr, sizeof(tmp_ip6));
__builtin_memcpy(&ip6->saddr, &ip6->daddr, sizeof(tmp_ip6));
__builtin_memcpy(&ip6->daddr, &tmp_ip6, sizeof(tmp_ip6));

/* Swap UDP ports */
__be16 tmp_port = udp->source;
udp->source = udp->dest;
udp->dest = tmp_port;
bump_stats();

return bpf_redirect(skb->ifindex, BPF_F_INGRESS);
```

1. DNS Header Rewrite

Sets the QR response bit and forces RCODE_REFUSED in DNS flags while recalculating checksum.

2. Reverse L2 & L3 Headers

Swaps Ethernet source/destination MACs and IPv6 addresses so frame routes back to origin.

3. Flip L4 UDP Ports

Exchanges UDP source and destination ports to target original client socket.

4. Re-inject Frame

Calls bpf_redirect(..., BPF_F_INGRESS) to instantly deliver modified frame directly.